Reactor Kinetics Lab Physics and Numerical Methods

1 Purpose

This note is the project-wide physics and numerics reference for the current *Reactor Kinetics Lab* repository. It explains how the implemented models are split across:

- the OpenMC MGXS export notebook `openmc/exportMGXS.ipynb`,
- the resolved multigroup diffusion notebook `theory/concentricModel.ipynb`,
- the exploratory one-group notebook `theory/reactorModel.ipynb`,

The notebooks are important **calibration and derivation workspaces**, while the Python backend and React frontend are the **runtime implementation**. The most important architectural points are:

> The repository now contains **two parallel modeling tracks**: an **OpenMC-informed resolved multigroup diffusion** pipeline that generates transport-derived, cell-homogenized group constants from Monte Carlo and solves the diffusion eigenvalue problem while preserving the exported cell regions, and an **interactive one-group web application** with a fast **point-kinetics surrogate** on the Overview page and a **spatially resolved 2D diffusion transient** on the Transient Diffusion page. The multigroup pipeline is an offline verification and future calibration path; its exploratory rod model is not yet the source of the web application's production rod-worth curve.

2 Role of the notebook

The notebook `theory/reactorModel.ipynb` studies a heavy-water moderated annular reactor with cylindrical symmetry in (r, z) and a top-inserted central control/shutdown bank. It is not the simulator itself; instead it is the place where the simplified reactor model is derived, calibrated, and sanity-checked before the resulting constants and algorithms are frozen into the production code. It has two estimation levels.

2.1 Estimate 1: homogeneous buckling sanity check

The first estimate is a coarse analytical model based on one-speed diffusion theory in a homogeneous bare cylinder:

$$-D\nabla^2\phi + \Sigma_a\phi = \frac{1}{k}\nu\Sigma_f\phi.$$

The criticality condition is written as

$$B_m^2 = \frac{\nu\Sigma_f - \Sigma_a}{D} = B_g^2,$$

with geometric buckling for a finite cylinder approximated by

$$B_g^2 \approx \left(\frac{2.405}{R}\right)^2 + \left(\frac{\pi}{H}\right)^2.$$

In the notebook this estimate is used only as a **first-pass size check**. It is intentionally approximate because it ignores:

- the inner moderator hole,
- reflector heterogeneity,
- explicit axial reflector slabs,
- and the geometric control rod.

2.2 Estimate 2: full 2D r–z finite-difference diffusion model

The second estimate is the notebook's main result. It solves a one-group diffusion eigenvalue problem on a cylindrical r–z mesh:

$$-\nabla \cdot (D\nabla\phi) + \Sigma_a\phi = \frac{1}{k_{\text{eff}}}\nu\Sigma_f\phi.$$

This estimate captures:

- radial and axial leakage,
- the annular fuel geometry,
- inner and outer D_2O regions,
- axial reflector slabs,
- and the real rod tip position as insertion changes.

The notebook uses this model to:

1. search for critical annular geometry,
2. compute the clean-core eigenvalue,
3. scan rod insertion versus reactivity,
4. and convert that scan into the calibration constants later frozen in the backend.

3 Geometry, materials, and assumptions

3.1 Core layout

The modeled reactor is an annular cylindrical core with a central D_2O region, a homogenized fuel annulus, and an outer D_2O reflector. The current frozen geometry in `backend/reactor_backend/calibration.py` is:

Quantity	Value
Inner moderator radius R_{inner}	80.0 cm
Fuel outer radius R_{fuel}	345.6 cm
Reflector outer radius R_{refl}	405.6 cm
Active height H	691.2 cm
Axial reflector thickness H_{refl}	60.0 cm per side
Equivalent rod radius r_{rod}	50.0 cm

The model is axisymmetric, so the mesh covers only $r \geq 0$ while still representing the full 3D core through cylindrical volume weighting $2\pi r\, dr\, dz$.

3.2 One-group effective constants

The notebook starts from microscopic cross sections at the 2200 m/s thermal reference point and then builds **engineering-effective** one-group constants. The production code uses the following frozen values:

Region	D [cm]	Σ_a [cm^{-1}]	$\nu\Sigma_f$ [cm^{-1}]
Inner D_2O moderator	1.25	3.5×10^{-4}	0
Fuel annulus	0.95	8.5×10^{-3}	8.59×10^{-3}
Outer D_2O reflector	1.15	4.0×10^{-4}	0

These are **homogenized one-group constants**, not a multigroup transport library. The fuel values intentionally fold together:

- natural uranium fuel,
- D_2O moderation inside the lattice,
- resonance/self-shielding effects,
- and spectrum averaging into one effective thermal group.

The control bank is represented as an added absorber in the inner moderator region with

$$\Delta\Sigma_{a,\text{rod,max}} = 0.25 \text{ cm}^{-1}.$$

This is likewise an effective one-group quantity rather than a detailed rod-cell transport model.

3.3 How the physics was manipulated into these constants

This project deliberately compresses more detailed reactor physics into a one-group annular model that can run interactively. The main reductions are:

1. **Energy collapse.** Multigroup slowing-down and spectral effects are represented by a single thermal group with effective $(D, \Sigma_a, \nu\Sigma_f)$.
2. **Lattice homogenization.** The fuel annulus is not modeled as explicit fuel pins in moderator. Instead natural uranium and D_2O are volume-averaged into one fissile ring region.

3. **Leakage folded into effective losses.** In a one-group model, any loss of neutrons to higher-energy groups or to unresolved geometric detail is represented as part of an effective absorption-like loss.

4. **Equivalent rod model.** The real control/shutdown system is collapsed to one central cylindrical absorber with an effective radius and an effective extra absorption $\Delta\Sigma_a$.

5. **Calibration by 2D diffusion, not by transport.** The final rod worth curve and critical insertion are taken from repeated diffusion eigenvalue solves, then frozen for the point model.

The notebook is explicit about these manipulations. For example, the D_2O "effective" Σ_a is much larger than the true 2200 m/s absorption of pure heavy water, because the one-group model uses Σ_a as a *net loss coefficient* after spectral and geometric simplification. Likewise, the fuel-annulus Σ_a is intentionally inflated relative to a naive 2200 m/s natural-uranium homogenization so the one-group annular model better matches the critical-size and rod-worth behavior seen in the notebook's 2D studies.

3.4 Boundary assumptions

The diffusion solves use extrapolated zero-flux boundaries rather than placing the computational boundary exactly at the physical reflector edge. In code this is represented by an extrapolation factor of $2.13D_{\text{refl}}$, so the radial and axial computational extents are larger than the physical core.

4 How the notebook feeds the web app

The notebook does *not* run the web app directly. Instead it establishes the physical basis that is later frozen into backend constants and reusable solver code:

1. **Estimate 1** gives an analytical reference for rough critical size.

2. **Estimate 2** performs the annular r–z diffusion study that supplies the production calibration.

3. The resulting geometry, material constants, and rod-worth data are copied into `calibration.py`.

4. The backend then exposes two kinds of runtime models:

    - a calibrated point-kinetics model for fast dashboard transients,
    - and spatial diffusion solvers for the Core and Transient pages.

So the notebook is best viewed as the **physics derivation and calibration workspace**, while the Python backend is the **production implementation**.

4.1 How the notebook output becomes backend input

The handoff from notebook to code is not symbolic; it is numerical. The production backend lifts the notebook's final values into `calibration.py`:

- annular geometry dimensions,
- one-group material constants,
- clean-core k_{eff} and excess reactivity,

- the 11-point rod-worth table,
- and the derived critical insertion percentage.

This means the backend no longer recomputes those design-calibration studies on startup. Instead it assumes the notebook has already answered the "what reactor are we modeling?" question and the Python solvers answer the "how does that reactor evolve for this input?" question.

5 Overview page: point-kinetics transient model

5.1 Governing equations

The Overview page uses the standard six-group delayed-neutron point-kinetics system:

$$\frac{dn}{dt} = \frac{\rho - \beta}{\Lambda} n + \sum_{i=1}^{6} \lambda_i C_i, \tag{1}$$

$$\frac{dC_i}{dt} = \frac{\beta_i}{\Lambda} n - \lambda_i C_i, \qquad i = 1, \ldots, 6. \tag{2}$$

Here:

- $n(t)$ is the normalized neutron population,
- ρ is reactivity in $\Delta k/k$,
- $\beta = \sum_i \beta_i$ is the effective delayed-neutron fraction,
- Λ is the prompt neutron generation time,
- C_i is precursor concentration for delayed group i,
- λ_i is the delayed-group decay constant.

5.2 Constants used by the web app

The current production constants in `config.py` are:

$$\beta_{\text{eff}} = 0.00651 = 651 \text{ pcm},$$
$$\Lambda = 5 \times 10^{-4} \text{ s}.$$

The six delayed groups are:

Group	β_i	λ_i [s^{-1}]
1	0.00025	0.0124
2	0.00138	0.0305
3	0.00122	0.1110
4	0.00264	0.3010
5	0.00075	1.1400
6	0.00027	3.0100

Displayed output is scaled from the nominal operating point

$$P_0 = 20 \text{ MW}_{\text{th}}, \qquad \Phi_0 = 1.5 \times 10^{12} \text{ n cm}^{-2} \text{s}^{-1},$$

using

$$P(t) = P_0 n(t), \qquad \Phi(t) = \Phi_0 n(t).$$

5.3 How rod motion enters the point model

The point-kinetics solver does not compute rod worth from first principles during runtime. Instead it reuses the **2D diffusion-derived rod-worth table** frozen in `calibration.py`. Reactivity is computed as

$$\rho_{\text{tot,pcm}}(x) = \rho_{\text{clean,pcm}} + \Delta\rho_{\text{rod,pcm}}(x) + \rho_{\text{scram,pcm}},$$

where $x \in [0, 1]$ is rod insertion fraction. In the current calibration:

$$\begin{aligned} k_{\text{clean}} &= 1.000199, \\ \rho_{\text{clean}} &= +19.9 \text{ pcm}, \\ x_{\text{crit}} &\approx 0.193, \\ \Delta\rho_{\text{rod}}(1.0) &= -164.8 \text{ pcm}, \\ \rho_{\text{scram}} &= -450 \text{ pcm when latched.} \end{aligned}$$

The backend interpolates linearly between 11 precomputed insertion points $x = 0.0, 0.1, \ldots, 1.0$.

5.4 Numerical method

The point-kinetics equations are stiff, so the backend does not use explicit Euler. Instead `engine.py` advances both the neutron population and the precursors implicitly at the new time level and then algebraically eliminates the precursor unknowns.

Start from the backward-Euler discretization

$$\frac{n^{k+1} - n^k}{\Delta t} = \frac{\rho - \beta}{\Lambda} n^{k+1} + \sum_{i=1}^{6} \lambda_i C_i^{k+1}, \tag{3}$$

$$\frac{C_i^{k+1} - C_i^k}{\Delta t} = \frac{\beta_i}{\Lambda} n^{k+1} - \lambda_i C_i^{k+1}. \tag{4}$$

Solving the precursor equation for the new precursor gives

$$C_i^{k+1} = \frac{C_i^k + \Delta t\,(\beta_i/\Lambda)\, n^{k+1}}{1 + \lambda_i \Delta t},$$

which is the relation already visible in the source.

Substituting that expression into the neutron balance yields

$$\frac{n^{k+1} - n^k}{\Delta t} = \frac{\rho - \beta}{\Lambda} n^{k+1} + \sum_i \lambda_i \frac{C_i^k + \Delta t\,(\beta_i/\Lambda)\, n^{k+1}}{1 + \lambda_i \Delta t}.$$

Now separate the terms involving only known old-state data from the terms that still multiply n^{k+1}:

$$\frac{n^{k+1} - n^k}{\Delta t} = \sum_i \frac{\lambda_i C_i^k}{1 + \lambda_i \Delta t} + \left[\frac{\rho - \beta}{\Lambda} + \sum_i \frac{\lambda_i \Delta t\, \beta_i/\Lambda}{1 + \lambda_i \Delta t} \right] n^{k+1}.$$

Rearranging gives one scalar solve for the new neutron population,

$$n^{k+1} = \frac{n^k + \Delta t \sum_i \dfrac{\lambda_i C_i^k}{1 + \lambda_i \Delta t}}{1 - \Delta t \dfrac{\rho - \beta}{\Lambda} - \Delta t \sum_i \dfrac{\lambda_i \Delta t\, \beta_i/\Lambda}{1 + \lambda_i \Delta t}}.$$

In the implementation this matches the code variables `delayed_source`, `delayed_coupling`, `numerator`, and `denominator` in `engine.py`. The precursor update then uses the newly computed n^{k+1}.

Numerically, this matters because the prompt part of the kinetics is very fast relative to the slowest delayed groups. A naive explicit step would require a much smaller Δt for stability, while the semi-implicit update remains well behaved for the chosen browser-friendly step size.

5.5 How the Overview transient is operationalized

The point-kinetics equations are not advanced continuously in a background thread. Instead `service.py` advances them *opportunistically* whenever the frontend polls or sends a command:

1. compute elapsed wall time since the last request,
2. cap that wall-time gap to avoid large jumps after browser pauses,
3. multiply by the configured time-scale factor,
4. then march the engine forward in repeated 0.02 s internal steps.

This architecture is a numerical simplification as much as a software one: the browser sees a continuous transient, but the backend actually performs a sequence of bounded substeps and samples history only every 0.25 s.

The current runtime tuning is:

Parameter	Value
Integrator step	0.02 s
History sampling	0.25 s
Time scale	8× wall clock
Frontend poll interval	100 ms
Automatic SCRAM threshold	26 MW_{th}

6 Core page: steady 2D diffusion solution

The Core page is the repository's **spatial steady-state neutronics view**. It reuses the same annular one-group model as the notebook and solves the 2D eigenvalue problem for the *current rod position* taken from the Overview simulation.

6.1 Discretization

The implementation in `diffusion.py` and `core_service.py` uses:

- a cell-centered finite-difference mesh in r and z,
- harmonic averaging of diffusion coefficients at material interfaces,
- sparse matrix assembly for leakage, absorption, and fission terms,
- and a power-iteration style eigenvalue solve for k_{eff} and the fundamental flux shape.

The production Core page solve runs at

$$\Delta r = \Delta z = 3 \text{ cm},$$

and results are cached by rod insertion state so repeated requests do not repeat the full eigenvalue solve.

6.2 How the steady diffusion matrices are built

The Python solver does not discretize the Laplacian in Cartesian form and then "pretend" the reactor is cylindrical. It assembles the operator directly in cylindrical coordinates. On each cell-centered mesh point (r_i, z_j) it builds:

- radial couplings weighted by the cylindrical face areas $r_{i\pm1/2}$,
- axial couplings weighted by $1/\Delta z^2$,
- and a diagonal absorption term from the local Σ_a.

Material interfaces are handled with the harmonic mean of adjacent diffusion coefficients, which is the standard finite-volume-like choice for discontinuous media because it preserves current continuity better than a simple arithmetic average.

The assembled linear system can be written schematically as

$$L\phi = \frac{1}{k} F\phi,$$

where:

- L is a sparse loss matrix containing leakage and absorption,
- F is a diagonal sparse matrix containing $\nu\Sigma_f$ in fuel cells and zero elsewhere.

The reason for that structure is direct. After cell-centered finite-difference discretization, the loss term at cell (i, j) depends only on:

- the flux in the same cell,
- the two radial neighbors $(i-1, j)$ and $(i+1, j)$,
- and the two axial neighbors $(i, j-1)$ and $(i, j+1)$.

So each row of L contains only a small five-point stencil worth of nonzero entries, making L sparse. By contrast, the fission source has no spatial derivative in this one-group model; it is simply

$$(F\phi)_{ij} = \nu\Sigma_{f,ij}\phi_{ij}.$$

That is a cell-local multiplication, so after flattening the 2D mesh into a single vector, F becomes diagonal.

In other words:

- **leakage/absorption** move neutrons between nearby cells or remove them locally $\rightarrow$ sparse coupling matrix L,
- **fission production** multiplies the flux only in the current cell $\rightarrow$ diagonal production matrix F.

6.3 How the steady eigenvalue solve works numerically

The function `solve_2d()` uses a source-iteration / power-iteration structure:

1. start from a positive flux guess ϕ and an initial $k = 1$,
2. compute the fission source $F\phi$,
3. solve the fixed loss system
$$L\phi^* = F\phi$$
with a sparse direct solver,
4. clip tiny negative roundoff values to zero,
5. normalize ϕ^* by its peak,
6. update the eigenvalue with the ratio of new to old total fission source,
$$k_{\text{new}} = \frac{\sum(F\phi^*)}{\sum(F\phi)},$$
7. and repeat until $|k_{\text{new}} - k| < \texttt{tol}$ after a few warmup iterations.

Two numerical choices are especially important:

- the loss matrix L is factorized once per solve with `scipy.sparse.linalg.factorized()`, so each iteration is just a triangular solve instead of a fresh matrix inversion;
- the flux is normalized by peak value, not by integral power, because the steady eigenproblem only needs the shape and eigenvalue.

6.4 How geometry and rod-worth calibration are solved

The notebook and backend use the steady diffusion solver in two further numerical loops:

1. **critical-radius search:** bisection on R_{fuel} until the solved k_{eff} brackets unity to within tolerance;
2. **rod-worth scan:** repeated solves at insertion fractions $x = 0, 0.1, \ldots, 1.0$, converting each k to reactivity $\rho = (k - 1)/k$ in pcm.

The rod-worth scan may warm-start each solve with the previous flux shape. That is a purely numerical acceleration: neighboring rod positions have similar flux shapes, so reusing the previous converged state reduces iteration count without changing the underlying model.

6.5 What the page returns

The backend returns:

- the peak-normalized 2D flux field $\phi(r, z)$,
- a radial profile through the midplane,
- an axial profile through the fuel-annulus midpoint,
- and metadata including k_{eff}, mesh spacing, and iteration count.

This is the web app's direct counterpart to the notebook's Estimate 2 flux maps.

7 Transient Diffusion page: direct time-dependent diffusion

7.1 Why this page exists

The Overview page assumes a fixed spatial shape and evolves only an amplitude. The Transient Diffusion page removes that simplification and evolves the **spatial flux field itself**, together with the six delayed-neutron precursor fields.

7.2 Governing equations

The transient diffusion solver in `backend/reactor_backend/transient_diffusion.py` solves

$$\frac{1}{v}\frac{\partial \phi}{\partial t} = -L\phi + (1-\beta)\frac{F}{k_{\text{ref}}}\phi + \sum_{i=1}^{6}\lambda_i C_i, \tag{5}$$

$$\frac{\partial C_i}{\partial t} = \beta_i \frac{\nu\Sigma_f}{k_{\text{ref}}}\phi - \lambda_i C_i, \qquad i = 1,\dots,6, \tag{6}$$

where:

- L is the spatial loss operator (leakage plus absorption),
- F is the diagonal fission-production operator,
- $C_i(r,z,t)$ is the precursor field for group i,
- $v = 2.2 \times 10^5$ cm/s is the one-group thermal neutron speed,
- k_{ref} is the eigenvalue of the initialization state.

The division by k_{ref} is deliberate: it references the transient fission source to the eigenvalue of the initialized state on the transient mesh. That keeps the transient solve anchored to the same discrete criticality reference and substantially reduces startup mismatch relative to using the raw unscaled fission operator.

7.3 Numerical method

The transient page uses **fully implicit Euler**. Eliminating the precursors gives one sparse linear solve per time step:

$$A\phi^{n+1} = b,$$

with

$$A = \frac{1}{v\Delta t}I + L - b_{\text{eff}}\,\text{diag}\left(\frac{\nu\Sigma_f}{k_{\text{ref}}}\right),$$

and

$$b_{\text{eff}} = (1-\beta) + \sum_i \frac{\beta_i \lambda_i \Delta t}{1+\lambda_i \Delta t}.$$

After ϕ^{n+1} is solved, each precursor field is updated by

$$C_i^{n+1} = \frac{C_i^n + \Delta t\,\beta_i(\nu\Sigma_f/k_{\text{ref}})\phi^{n+1}}{1+\lambda_i\Delta t}.$$

This scheme is unconditionally stable for the linear model and is much better suited than explicit Euler to delayed-neutron reactor kinetics.

In implementation terms, the transient solver performs three numerically distinct operations:

1. **Initialization:** run a steady `solve_2d()` eigenvalue solve on the transient mesh, flatten the 2D flux field, and initialize each precursor field with the steady relation

$$C_{i,0} = \frac{\beta_i}{\lambda_i} \frac{\nu\Sigma_f}{k_{\text{ref}}} \phi_0.$$

2. **Matrix rebuild on rod motion:** if rod insertion changes, rebuild the sparse matrix A and refactorize it. In code this cache is keyed by the rounded rod fraction together with k_{ref}.

3. **Cheap repeated stepping between rod changes:** if rod insertion is unchanged, reuse the cached LU factorization so each new time step is just one sparse linear solve plus explicit vector updates for the six precursor groups.

This split is why the transient page can remain interactive even though it is a full spatial solver: the expensive part is the factorization, not the individual post-factorization time steps.

7.4 How the transient physics is manipulated into a solvable model

The transient page is more detailed than point kinetics, but it is still not a full reactor multiphysics model. Several physics reductions are built into the solver:

- prompt and delayed neutrons are treated in one energy group,
- thermal-hydraulic feedback is absent, so all cross sections stay fixed while the transient runs,
- rod motion changes only the absorber map inside the central control-bank cylinder,
- the internally stored flux is an *absolute amplitude field*, while UI heatmaps are separately peak-normalized display copies,
- and power is derived from a cylindrical fission integral rather than from a standalone thermal-hydraulic power balance.

That last point is numerically important: the solver never renormalizes its internal state after each step. If it did, the transient could not display real growth or decay in P/P_0.

7.5 Runtime settings

To keep the direct transient responsive in a browser-driven local app, the production solver uses:

Parameter	Value
Transient mesh	$\Delta r = \Delta z = 5$ cm
Time step	$\Delta t = 1$ s
Display heatmap	40×60 points
History limit	200 points

The transient mesh is intentionally coarser than the Core page mesh so the LU factorization can be rebuilt quickly whenever rod insertion changes.

7.6 Service-level time stepping

The transient page is advanced by `transient_service.py`, which keeps one shared solver instance and one shared state vector. On every `GET /api/transient-diffusion/state` request it:

1. measures elapsed wall time,
2. converts that to simulated time at a fixed scale of $1\times$,
3. computes how many whole 1 s transient steps fit in that interval,
4. caps the advance to at most four steps per request,
5. and appends one history sample per accepted time step.

This cap is not part of the physics model; it is a numerical and UX safeguard so one delayed frontend poll cannot trigger an unexpectedly long blocking solve.

7.7 Power normalization

The transient diffusion page no longer assumes that power is proportional to one global amplitude alone. Instead it evaluates the cylindrical fission-rate integral

$$P \propto \sum_j \nu\Sigma_{f,j}\,\phi_j\,V_j, \qquad V_j \propto r_j \Delta r \Delta z,$$

and reports

$$\frac{P}{P_0} = \frac{\sum_j \nu\Sigma_{f,j}\phi_j V_j}{\sum_j \nu\Sigma_{f,j}\phi_{j,0} V_j}.$$

The factor 2π cancels in the ratio, so the implementation only needs the $r_j \Delta r \Delta z$ weight per cell.

8 OpenMC-informed resolved multigroup diffusion modeling

The one-group annular model described above is intentionally simplified for interactive use. In parallel, the repository now includes a **resolved multigroup diffusion** modeling chain that derives its nuclear data directly from an OpenMC continuous-energy Monte Carlo transport calculation. This section describes that higher-fidelity pipeline.

8.1 Overview of the pipeline

The multigroup workflow has four stages, all implemented as reusable Python modules rather than notebook-only logic:

1. **MGXS export** (`openmc/mgxs_export.py`): attach multigroup cross-section tallies to each resolved OpenMC cell (fuel rings, central channel, coolant, moderator, reflector, control rod), run an eigenvalue calculation, and write group constants plus delayed-neutron data as JSON and CSV.
2. **Adapter and region mapping** (`backend/reactor_backend/openmc_mgxs_adapter.py`): parse the OpenMC `model.xml` to extract cell boundaries (z-cylinders and z-planes), load the canonical `mgxs_constants.json`, validate the scattering contract, preserve the centimeter-based OpenMC units, and map each OpenMC cell to a `MultiGroupRegion` inside a `CylindricalLayeredMode`

3. **Finite-volume multigroup diffusion solver** (`backend/reactor_backend/multigroup_diffusion.py`): assemble per-group sparse spatial operators, solve the multigroup eigenvalue problem, and optionally scan rod insertion.

4. **Persistent preparation cache** (`backend/reactor_backend/multigroup_diffusion_cache.py`): fingerprint the MGXS data, mesh settings, and solver configuration; build CSR matrices and a clean-core solution once, then persist everything to disk for fast reload.

The user-facing entry point is the notebook `theory/concentricModel.ipynb`, which selects an export directory, configures mesh spacing and solver tolerances, triggers cache preparation, and plots the resulting flux and rod-worth scan.

8.2 MGXS export from OpenMC

The export module (`mgxs_export.py`) takes a previously built OpenMC `model.xml` and attaches multigroup tally objects to every resolved cell domain. The domain definitions are configurable; the resolved mode maps each `fuel_ring_N` cell to its own domain label, plus the central moderator channel, heavy-water coolant, outer moderator, reflector, and parked control rod. A frozen `MGXSExportConfig` dataclass captures the particle count, batch schedule, energy-group edges (any CASMO structure from 1 to 70 groups), Legendre scattering order (P0, P1, P3, or P5), and the delayed-neutron group count.

Two scattering representations are tallied from the same continuous-energy run:

- The **canonical diffusion export** uses non-ν, uncorrected scattering (`scatter_correction = null`). The adapter prefers the `consistent scatter matrix` and falls back to `scatter matrix`. Only the zeroth Legendre moment is passed to diffusion; higher moments are recorded and discarded.

- A **supplemental P0-corrected consistent scattering library** is tallied for OpenMC's own multigroup transport validation run. This correction is validation-only and is not written into the canonical diffusion JSON.

When `validation_scatter_correction = P0`, the export contract requires `legendre_order = 0`, because OpenMC does not apply that correction to a higher-order Legendre library. A higher-order canonical export can still be read by the adapter, but the validation correction must be disabled for that run and the diffusion path will retain only moment zero.

The diffusion coefficient is taken directly from OpenMC's transport cross-section tally:

$$D_g = \frac{1}{3\,\Sigma_{\mathrm{tr},g}}.$$

This distinction is important. OpenMC's P0 scatter correction modifies the isotropic scattering data used by its multigroup *transport* calculation to represent part of the first angular moment. The diffusion calculation instead represents anisotropic migration through $\Sigma_{\mathrm{tr},g}$ in D_g. Its scattering source therefore uses the uncorrected non-ν P0 reaction-rate matrix. The two calculations use different angular closures; agreement of the OpenMC multigroup validation does not imply that the diffusion approximation has been calibrated.

All neutronics quantities remain in OpenMC's native centimeter-based units: D in cm, macroscopic cross sections in cm^{-1}, and geometry and mesh dimensions in cm. No SI conversion is performed in the adapter. This gives $D\nabla^2\phi$ and $\Sigma\phi$ consistent units throughout the steady solver.

Each export run produces:

- `mgxs_constants.json` — nested group constants, delayed-neutron data, and metadata for every domain;
- `group_constants.csv` — flat table of all group-constant rows;
- `delayed_neutrons.csv` — flat table of all delayed-neutron rows.

The notebook `openmc/exportMGXS.ipynb` runs group-count and Legendre-order sweeps, storing each result under `group_sweep/group_<N>/`. It also runs an OpenMC multigroup validation step that rebuilds an `mgxs.h5` library and solves the same geometry in multigroup transport mode, producing the $\Delta\rho$ (MG − CE) values reported in the workflow document.

8.3 Adapter: from OpenMC cells to diffusion regions

The adapter (`openmc_mgxs_adapter.py`) is the bridge between the OpenMC export and the diffusion solver. It performs five operations:

1. **Geometry extraction.** Parses the OpenMC `model.xml` with Python's `ElementTree`, identifies every named cell, and extracts its radial and axial bounds from the z-cylinder and z-plane surfaces referenced in the cell's region specification.
2. **Contract validation.** Requires `scatter_correction = null` in the export config so the P0 scattering diagonal is not transport-corrected. Rejects exports that are missing required domains (fuel rings, central channel, moderator, reflector) or whose MGXS domain labels do not match the XML cell names.
3. **Scattering-matrix selection.** Prefers `consistent scatter matrix`; falls back to `scatter matrix`. Discards any Legendre moments above P0 and interprets the stored matrix as $\Sigma_{s,g'\to g}$, with the first group index denoting the source group and the second the destination group.
4. **Diffusion-coefficient validation.** For every active region/group pair, checks that the exported coefficient is finite, positive, and consistent with $D_g = 1/(3\Sigma_{\mathrm{tr},g})$ to numerical precision.
5. **Inactive-group sanitization.** Energy groups with zero absorption, zero fission, zero chi, and zero scatter coupling are flagged as inactive. Their undefined diffusion coefficients are replaced with a finite coefficient from another active group so the spatial operator remains well-posed; every replacement is reported in the adapter summary.

The output is a `CylindricalLayeredModel2D` containing one `MultiGroupRegion` per OpenMC cell. Each region holds per-group vectors for D, Σ_a, $\nu\Sigma_f$, χ, and a $G \times G$ P0 scattering matrix $\Sigma_{s,g'\to g}$. The radial mesh is fitted exactly to all XML-derived material boundaries so no interface is lost to grid coarsening. "Resolved" here means that no new cross-cell homogenization is introduced: each exported cell receives its own constants. The constants are still flux-weighted averages within that OpenMC tally cell, so this is not a pin-resolved or continuously varying material description.

8.4 Finite-volume multigroup diffusion solver

The solver in `multigroup_diffusion.py` solves the G-group steady-state diffusion eigenvalue problem on a 2D cylindrical (r, z) mesh:

$$-\nabla \cdot (D_g \nabla \phi_g) + \Sigma_{a,g}\, \phi_g + \sum_{g' \neq g} \Sigma_{s,g \to g'}\, \phi_g \;=\; \frac{\chi_g}{k_{\text{eff}}} \sum_{g'} \nu \Sigma_{f,g'}\, \phi_{g'} \;+\; \sum_{g' \neq g} \Sigma_{s,g' \to g}\, \phi_{g'}.$$

The left-hand side collects leakage, absorption, and outscatter from group g into a single loss operator A_g. The right-hand side contains the fission source (scaled by $1/k_{\text{eff}}$) and in-scatter from all other groups. Within-group scattering does not appear explicitly because its loss and gain terms cancel in the group balance. Thus the removal coefficient used in A_g is

$$\Sigma_{r,g} = \Sigma_{a,g} + \sum_{g' \neq g} \Sigma_{s,g \to g'}.$$

8.4.1 Spatial discretization

The mesh is a nonuniform, cell-centered finite-volume grid in cylindrical coordinates. Every material radius and axial plane extracted from `model.xml` is inserted as a mesh face before the intervals are subdivided. The target radial spacing is region dependent (fuel, core coolant, moderator, and reflector), while one target axial spacing is currently used throughout.

Integrating the diffusion equation over cell (i, j) gives a balance between currents through four faces and volumetric reaction terms. Omitting the common factor 2π, the cell volume is

$$V'_{ij} = \frac{1}{2}\left(r^2_{i+1/2} - r^2_{i-1/2}\right) \Delta z_j.$$

For two adjacent cells P and N, the face conductance is

$$H_{PN} = \left(\frac{\delta_P}{D_P} + \frac{\delta_N}{D_N}\right)^{-1},$$

where δ_P and δ_N are the center-to-face distances. This is the harmonic interface treatment implied by continuity of normal current. After division by V'_{ij}, the off-diagonal coefficient is the negative face conductance multiplied by the cylindrical face area. For example,

$$C_{i+1/2,j} = \frac{r_{i+1/2} H_{i,i+1}}{\frac{1}{2}(r^2_{i+1/2} - r^2_{i-1/2})},$$

and the corresponding axial coefficient is $C_{i,j+1/2} = H_{j,j+1}/\Delta z_j$. The diagonal contains the sum of all outgoing leakage coefficients plus $\Sigma_{r,g}$.

The assembled per-group loss operator A_g is a sparse CSR matrix with a five-point stencil in (r, z). It is factorized once per solve with a sparse direct solver (SuperLU via `scipy.sparse.linalg.factorized`).

8.4.2 Boundary conditions

At $r = 0$, the missing west-face term imposes the cylindrical symmetry condition

$$\left.\frac{\partial \phi_g}{\partial r}\right|_{r=0} = 0.$$

At the outer radial, upper axial, and lower axial boundaries, the solver imposes zero scalar flux at the computational boundary using the half-cell distance from the last cell center to the boundary face. The computational domain is extended beyond the physical reflector by

$$d_{\text{ext}} = 2.13 \max_g D_{\text{refl},g},$$

radially and at each axial end, and the extension is assigned reflector properties. Consequently the implemented condition is an approximate extrapolated-vacuum boundary:

$$\phi_g = 0 \quad \text{at the extrapolated boundary.}$$

The same extrapolation distance is used for every group because it is based on the maximum reflector diffusion coefficient. This is a pragmatic closure, not a group-dependent transport boundary condition; its sensitivity should be separated from spatial-mesh and MGXS-condensation errors.

8.4.3 Eigenvalue iteration

The multigroup eigenvalue problem is solved by nested iteration:

1. **Outer power iteration.** Start from a positive flux guess and an initial $k_{\text{eff}} = 1$. At each outer iteration:
   (a) compute the fission density $f = \sum_g \nu\Sigma_{f,g}\,\phi_g$;
   (b) form the fixed fission source $s_g^{\text{fiss}} = \chi_g\, f/k_{\text{eff}}$;
   (c) solve the coupled scattering system $A_g\phi_g = s_g^{\text{fiss}} + \sum_{g'\neq g} \Sigma_{s,g'\to g}\,\phi_{g'}$ for all groups.
2. **Inner scattering iteration.** The coupled system first receives at most five Gauss–Seidel sweeps over destination groups. The requested inner tolerance is scheduled from 10^{-2} in early outer iterations to the user-specified `inner_tol`.
3. **Matrix-free GMRES fallback.** If those Gauss–Seidel sweeps do not satisfy the active inner tolerance, the code solves the coupled scattering system with matrix-free GMRES, preconditioned by the same per-group SuperLU factors. This avoids assembling a monolithic $(G \times N_{\text{cells}}) \times (G \times N_{\text{cells}})$ matrix, which would produce prohibitive fill-in for higher group counts and fine meshes.
4. **Eigenvalue update.** After each outer iteration the eigenvalue is updated from the ratio of new to old total fission production,
$$k_{\text{new}} = k_{\text{old}}\,\frac{\sum_{i,j} V_{ij}\ \sum_g \nu\Sigma_{f,g,ij}\,\phi_{g,ij}^{\text{new}}}{\sum_{i,j} V_{ij}\ \sum_g \nu\Sigma_{f,g,ij}\,\phi_{g,ij}^{\text{old}}}.$$
5. **Flux normalization.** The flux is normalized so the volume-integrated fission production rate equals unity,
$$\sum_{i,j} V_{ij}\ \sum_g \nu\Sigma_{f,g,ij}\,\phi_{g,ij} = 1.$$
This fixes the otherwise arbitrary eigenvector scale and improves numerical conditioning. It is not an absolute power normalization; conversion to physical flux or power requires a separate amplitude based on reactor power and recoverable energy per fission.

Convergence is declared only after the final requested inner tolerance is active and both

$$\frac{|k_{\text{new}} - k_{\text{old}}|}{|k_{\text{new}}|} < \texttt{tol}$$

and the volume-weighted change in fission-source shape is below `source_tol`. A volume-weighted equation-balance residual is computed after convergence and reported as a diagnostic, but it is not currently a stopping criterion.

8.4.4 Rod-worth scan

The equivalent-absorber rod model adds a scalar $\Delta\Sigma_a$ inside the physical rod radius above an axially moving rod tip. In those cells it also sets $\nu\Sigma_f$, χ, and scattering to zero. This is an equivalent black/absorbing-region perturbation, not a material replacement using the parked-rod MGXS. A rod scan sweeps the insertion fraction $x \in [0, 1]$, rebuilding and factorizing the affected group operators at each position. The previous converged flux may be used as the next initial guess.

The resulting $\Delta\rho(x)$ is explicitly exploratory. It is not a transport-calibrated rod-worth curve because there are no rodded MGXS, SPH factors, or discontinuity factors in the present workflow. It should not be frozen into the web application without a separate validation study.

8.5 Persistent preparation cache

The full solve chain (adapter → mesh construction → per-group CSR assembly → convergence) is expensive, especially for higher group counts and fine meshes. The cache module (`multigroup_diffusion_cache.p`) avoids recomputing this work on every notebook restart or explicit cache preparation.

The cache fingerprint is a SHA-256 hash of:

- the canonical `mgxs_constants.json` contents,
- the colocated `model.xml` contents,
- the mesh-spacing configuration (target Δr and Δz per region),
- the rod-absorption parameter and solver tolerances,
- and an explicit cache-schema version.

The fingerprint does not hash the Python source itself. A numerical algorithm change that affects cached data therefore requires incrementing the cache schema version.

On a cache hit, the following are loaded directly from disk:

- radial and axial mesh edge arrays,
- cell-wise material arrays (D, Σ_a, $\nu\Sigma_f$, χ, and P0 scattering),
- one CSR spatial matrix per energy group,
- and the converged clean-core eigenvalue and flux field.

SuperLU factorization objects are process-local and are not serialized; a process that needs to solve again (e.g. for a rod scan) loads the CSR matrices and re-factorizes them in memory. The persisted operators describe the clean state only. A changed rod position alters the absorption and scattering maps, so its operators must still be rebuilt and factorized. On a cache miss, the full clean-state build-and-solve pipeline runs synchronously and the results are persisted.

The default cache root is `backend/.cache/openmc_diffusion/`. The CLI entry point `backend/openmc_to_dif` exposes a `-prepare-cache` flag for use outside the notebook.

8.6 Integration with the web application

The multigroup diffusion pipeline currently operates as an offline analysis and verification tool. A defensible path to the web backend is:

1. run the MGXS export once per group structure of interest,
2. prepare the diffusion cache from the chosen export,
3. establish clean-core mesh and group convergence against OpenMC transport,
4. introduce and validate SPH/discontinuity corrections if the intended accuracy requires them,
5. generate rodded MGXS or otherwise validate a rod-equivalence model,
6. only then expose qualified clean-core and rod-worth data to the backend.

The cache is still useful to a future backend because it can load a qualified clean solution and its CSR operators without re-running OpenMC. It does not, by itself, make rod-position sweeps cheap because each rod state changes the operators. Additional operator-reuse or low-rank-update work would be needed for that capability.

8.7 Current status and verification

The adapter and export configuration support the listed 1, 2, 4, 8, 16, 25, 40, and 70 group structures. The documented diffusion benchmark table currently contains resolved clean-core results through 16 groups; support for a group structure should not be confused with a completed mesh/runtime verification at that group count. The workflow document `theory/diffusion_mgxs_workflow.md` records the current comparison:

- OpenMC multigroup transport with the validation-only P0 correction approaches the continuous-energy reference as the group structure is refined: the reported MG biases are +806, +426, +166, and +66 pcm for 2, 4, 8, and 16 groups, respectively.
- The uncorrected diffusion bias (diffusion k_{eff} minus OpenMC MG k_{eff}) stabilizes around -400 to -500 pcm for the documented 4, 8, and 16 group cases. This is evidence of a repeatable residual bias, not proof that diffusion closure alone is its cause.
- The resolved two-group diffusion result is anomalous relative to that trend: its bias is about +1194 pcm relative to two-group OpenMC MG. This points to sensitivity to the very broad energy condensation and groupwise diffusion coefficients rather than a simple monotonic group-count convergence law.
- The resolved one-group diffusion result carries an approximately +1800 pcm bias relative to continuous energy. Because the fuel rings remain spatially resolved in this calculation, that result cannot be attributed to collapsing the rings into one spatial region.

The current acceptance criterion is trend and numerical sanity, not a tight pcm agreement target. SPH factors, discontinuity factors, and rodded MGXS are deferred until the uncorrected clean-core baseline is stable and documented.

9 How the modeling layers relate

Layer	Model	Purpose
OpenMC MGXS export	Resolved-cell multigroup	Generate transport-derived group constants and scattering matrices from continuous-energy Monte Carlo
Multigroup diffusion solver	G-group 2D finite-volume	Solve the resolved-cell clean-core eigenvalue problem; provide flux shapes and an exploratory equivalent-rod scan
Notebook estimate 1	Homogeneous buckling	First-pass analytical check of scale and leakage trends
Notebook estimate 2 / Core page	Steady 2D one-group diffusion	Compute k_{eff}, flux maps, and rod-worth calibration from effective constants
Overview page	0D six-group point kinetics	Fast transient driven by the calibrated reactivity curve
Transient Diffusion page	2D diffusion + 6 precursor fields	Spatial transient with evolving flux shape and power

The conceptual workflow is therefore:

1. Run OpenMC with resolved-cell MGXS tallies; export constants as JSON,
2. parse the export through the adapter and build a boundary-fitted finite-volume mesh,
3. solve and verify the clean-core multigroup diffusion eigenvalue problem (via `concentricModel.ipynb`),
4. treat the present equivalent-rod scan as exploratory until rodded data and diffusion corrections are validated,
5. qualify any future constants or reactivity curves before transferring them into the backend calibration module,
6. use the existing fast point model for dashboard-style operation on the Overview page,
7. and use the diffusion transient (one-group or, in the future, multigroup) when spatial dynamics matter.

10 Important assumptions and limitations

The one-group web-application models remain intentionally simplified:

- one energy group only,
- no thermal-hydraulic feedback,
- no fuel temperature, moderator temperature, void, xenon, or burnup effects,
- homogenized fuel and rod regions,

- heavy-water moderation is represented through effective diffusion and absorption constants rather than coupled moderator state equations.

The OpenMC-informed multigroup diffusion pipeline lifts the one-group limitation (the configured CASMO structures contain up to 70 groups) and avoids any additional cross-cell homogenization by preserving every exported OpenMC cell as a separate diffusion region. It still uses flux-weighted cell-homogenized constants and does not yet include:

- SPH factors or discontinuity factors to correct the diffusion transport approximation,
- rodded MGXS (the rod scan still uses an equivalent absorber),
- thermal-hydraulic or burnup feedback,
- time-dependent multigroup solves in the web backend.

So the repository should be read as an **educational reactor simulator with increasing levels of neutronics fidelity**, not as a licensing-grade core code.

11 Bottom line

The repository currently implements a hierarchy of reactor models with increasing spatial and spectral fidelity:

- At the top, **OpenMC continuous-energy Monte Carlo** provides reference k_{eff} values and generates resolved-cell multigroup cross sections.
- A **resolved multigroup diffusion solver** consumes those MGXS exports, solves the G-group eigenvalue problem on a boundary-fitted cylindrical mesh, and produces clean-core flux shapes plus an exploratory equivalent-rod response. Group refinement reduces energy-condensation error in the MGXS representation, but the remaining diffusion bias is not yet corrected by SPH or discontinuity factors.
- The older **one-group annular diffusion model** remains the calibration workbench for the interactive web app, providing effective constants and rod-worth data for the point-kinetics and steady/transient diffusion pages.
- The **point-kinetics transient** on the Overview page gives fast control-response exploration in the browser.
- The **direct diffusion transient** on the Transient Diffusion page captures evolving spatial flux shapes during rod motion.

The notebook `theory/concentricModel.ipynb` is the user-facing interface to the multigroup diffusion pipeline: select an MGXS export, prepare the cache, inspect the clean-core flux, and optionally run the uncalibrated equivalent-rod scan. The `diffusion_mgxs_workflow.md` document records the group-by-group verification results and the scattering contract that ties the export format to the solver expectations.

That is the current modeling hierarchy of the project.